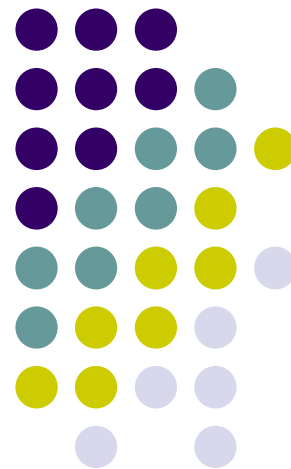


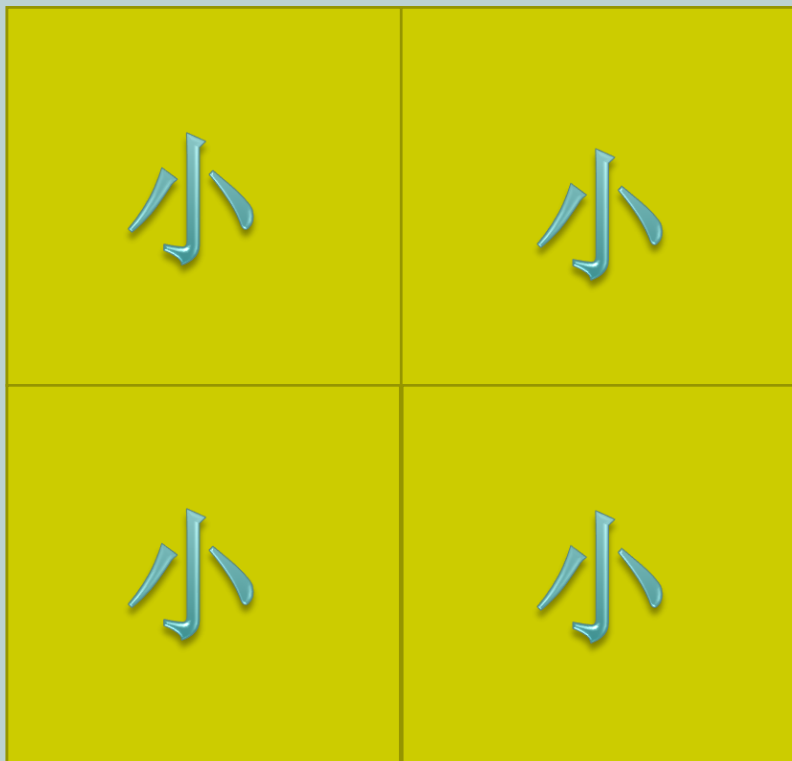
第2章 递归与分治策略 (二)

曾华琳



分治策略

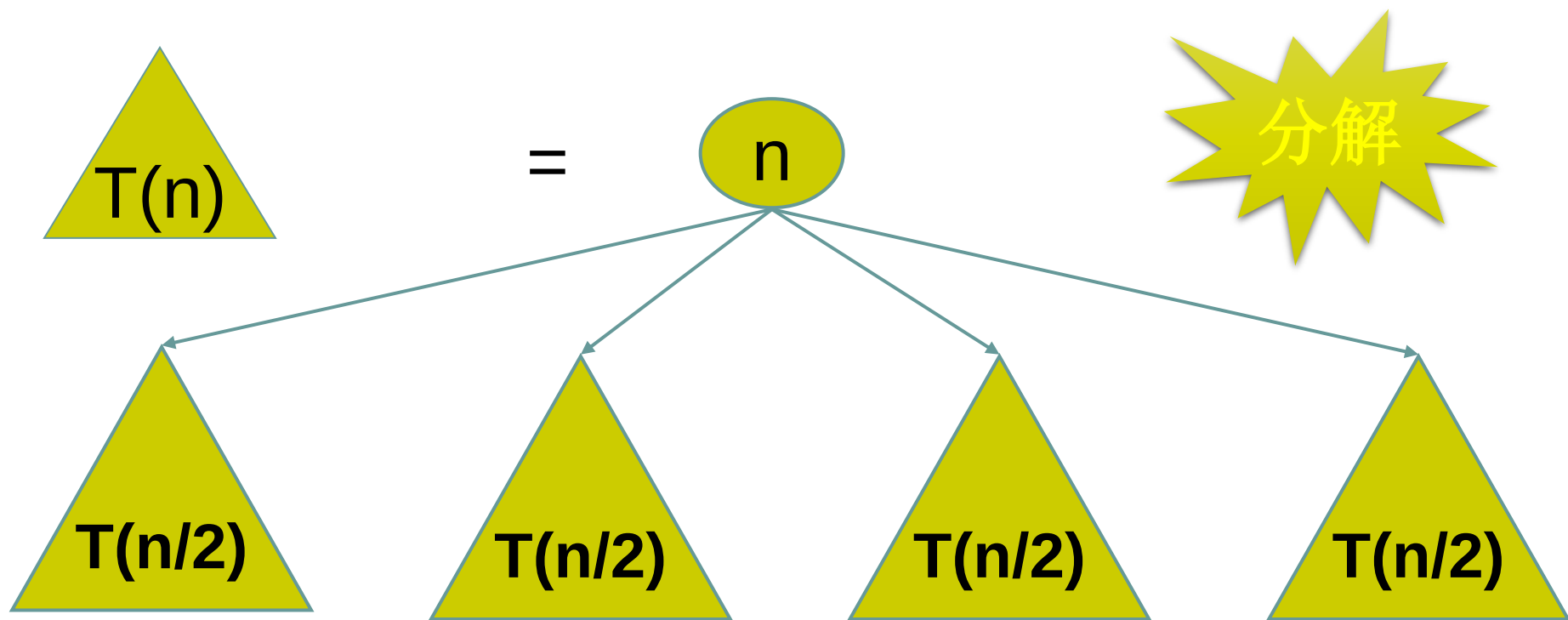
分解





分治算法总体思想

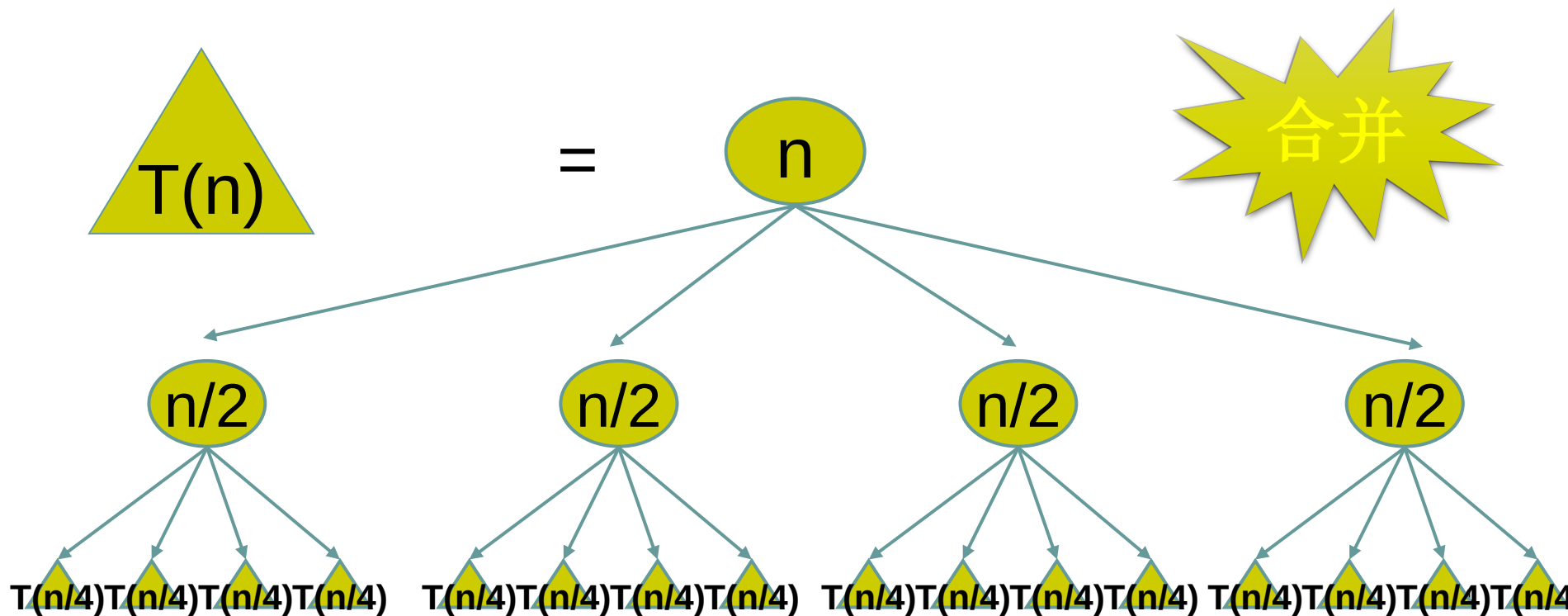
- 对这 k 个**子问题分别求解**。如果子问题的规模仍然不够小，则再划分为 k 个子问题，如此**递归**的进行下去，直到问题**规模足够**小，很容易求出其解为止。



分治算法总体思想



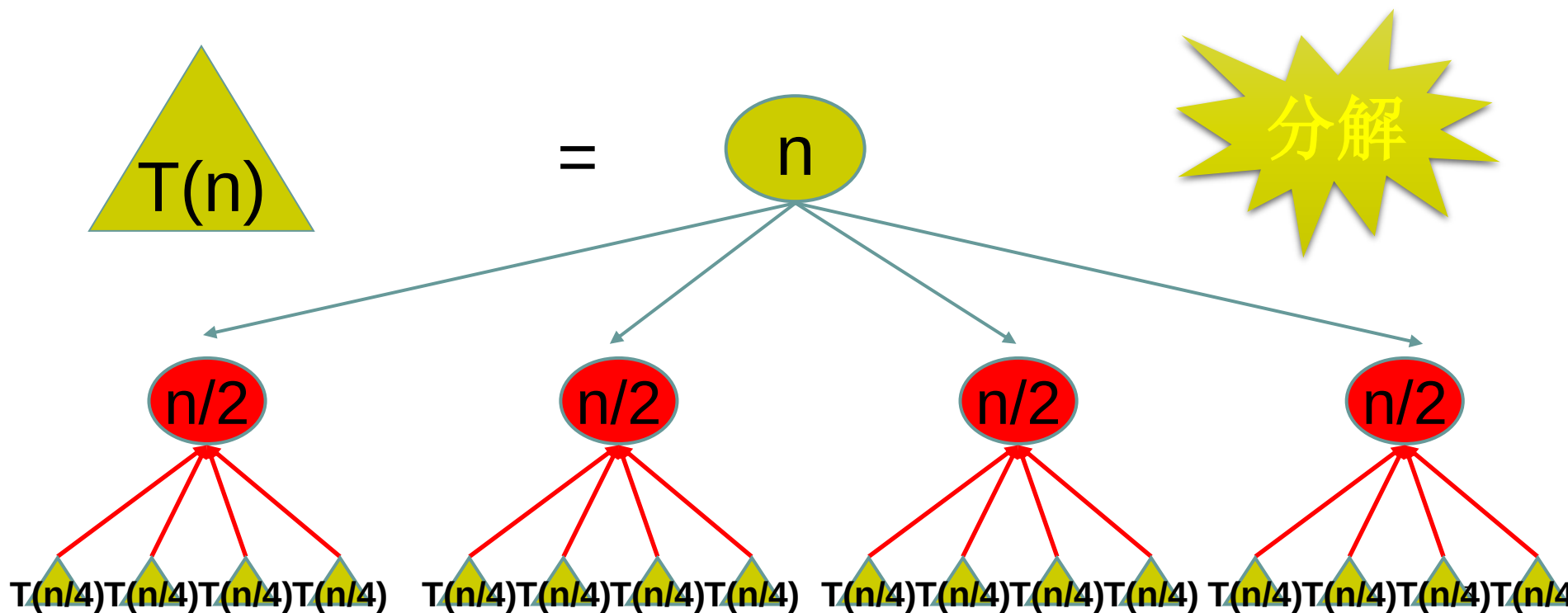
- 将求出的小规模的问题的解**合并**为一个更大规模的问题的解，**自底向上**逐步求出原来问题的解。



分治算法总体思想



- 将求出的小规模的问题的解**合并**为一个更大规模的问题的解，**自底向上**逐步求出原来问题的解。



分治算法总体思想



- 将求出的小规模的问题的解**合并**为一个更大规模的问题的解，**自底向上**逐步求出原来问题的解。

一个难以直接
解决的大问题



一些规模较小
的相同问题

各个击破
分而治之

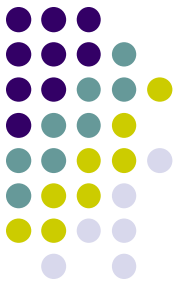


分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有**最优子结构性质**
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用**动态规划**较好。

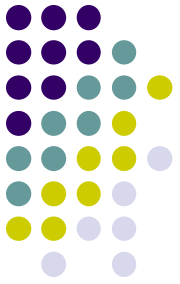


分治法的基本步骤

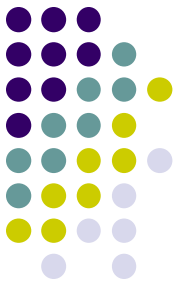
divide-and-conquer(P)

```
{  
  if ( | P | <= n0) adhoc(P); //解决小规模的问题  
  divide P into smaller subinstances P1,P2,...,Pk; //分解问题  
  for (i=1,i<=k,i++)  
    yi=divide-and-conquer(Pi); //递归的解各子问题  
  return merge(y1,...,yk); //将各子问题的解合并为原问题的解  
}
```

人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的k个子问题的处理方法是行之有效的。这种使子问题规模大致相等的做法是出自一种**平衡(balancing)子问题**的思想，它几乎总是比子问题规模不等的做法要好。



二分搜索技术

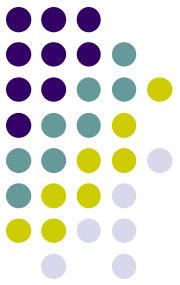


二分搜索技术

给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。

- 分析：**
- ✓ 该问题的规模缩小到一定的程度就可以容易地解决；
 - ✓ 该问题可以分解为若干个规模较小的相同问题；
 - ✓ 分解出的子问题的解可以合并为原问题的解；
 - ✓ 分解出的各个子问题是相互独立的。

分析：很显然此问题分解出的子问题相互独立，即在 $a[i]$ 的前面或后面查找 x 是独立的子问题，因此满足分治法的第四个适用条件。



二分搜索技术

给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。

据此容易设计出**二分搜索算法**：

```
template<class Type>
int BinarySearch(Type a[], const Type& x,
{
    while (r >= l){
        int m = (l+r)/2;
        if (x == a[m]) return m;
        if (x < a[m]) r = m-1; else l = m+1;
    }
    return -1;
}
```

算法复杂度分析：

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂性为 $O(\log n)$ 。



合并排序



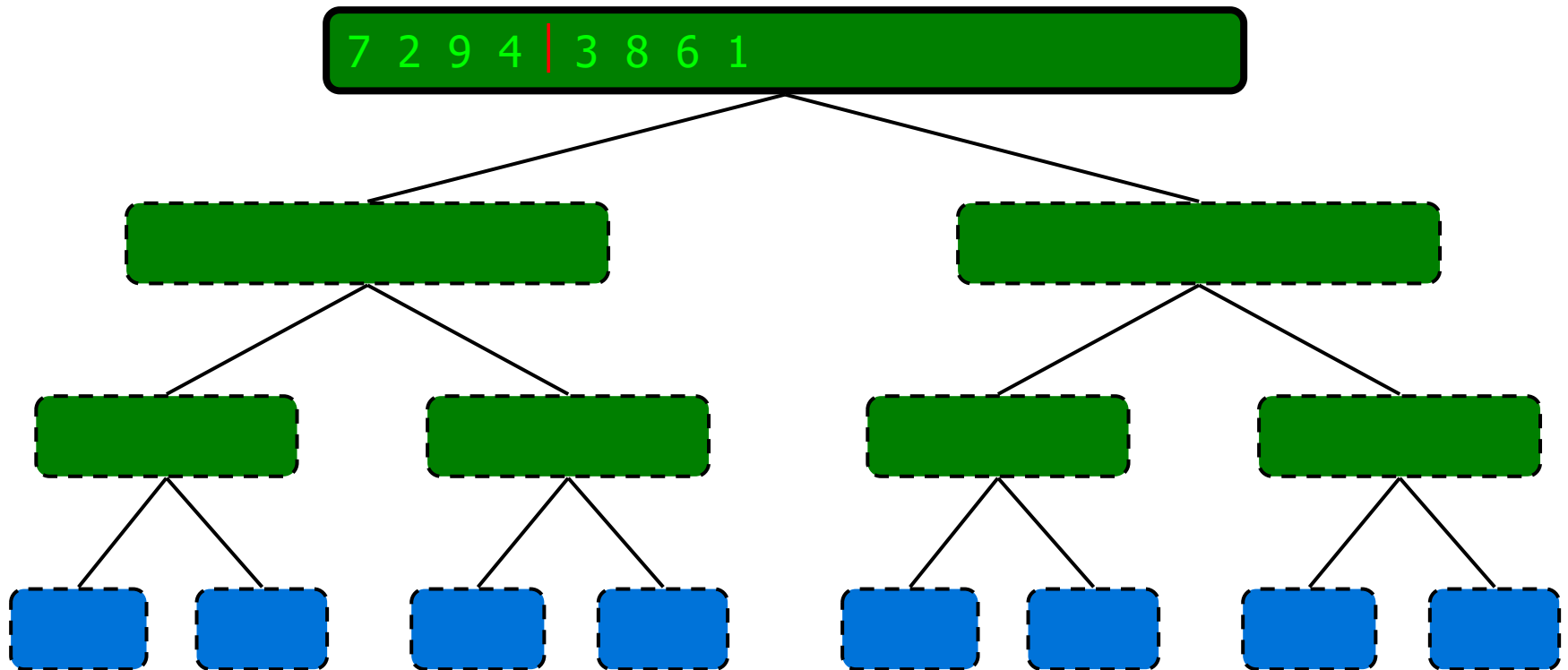
合并排序

基本思想： 将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。

```
void MergeSort(Type a[], int left, int right)
{
    if (left < right) { //至少有2个元素
        int i = (left + right) / 2; //取中点
        mergeSort(a, left, i);
        mergeSort(a, i + 1, right);
        merge(a, b, left, i, right); //合并到数组b
        copy(a, b, left, right); //复制回数组a
    }
}
```

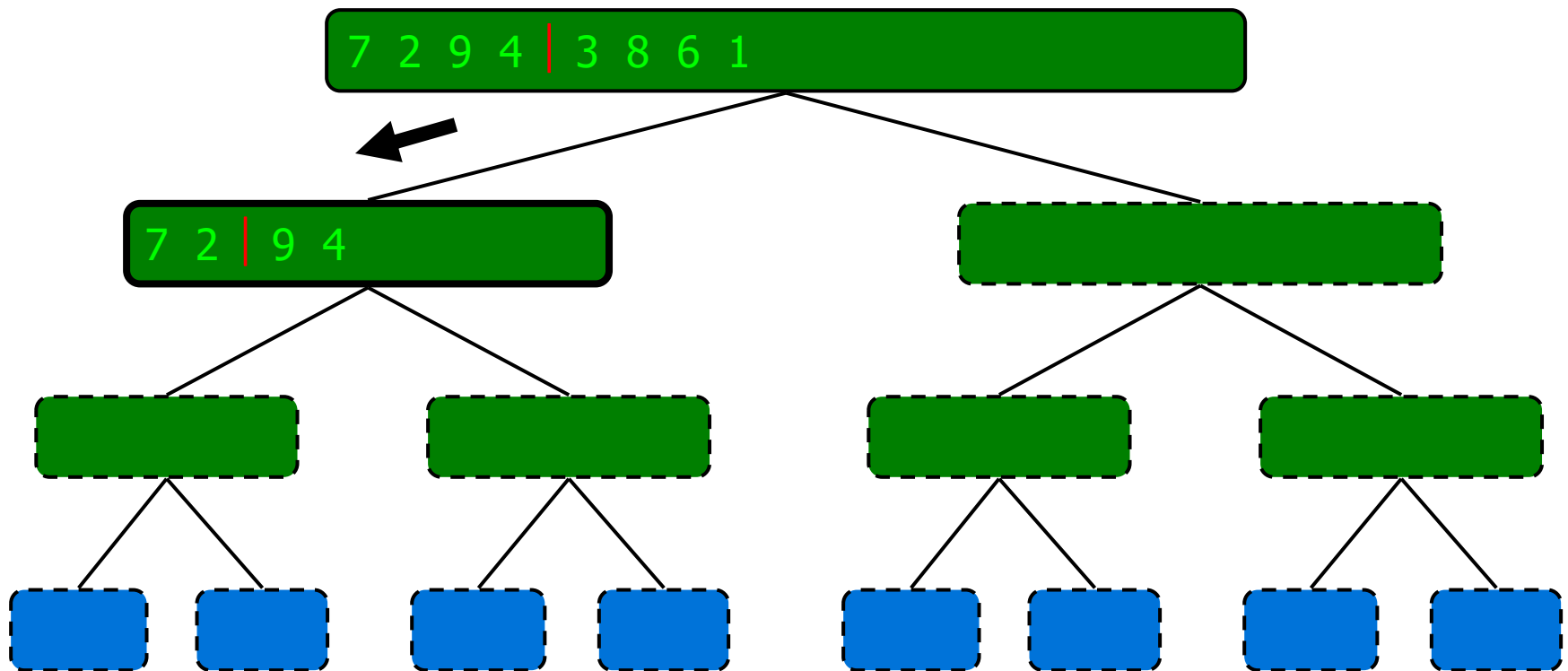
Execution Example

- Partition



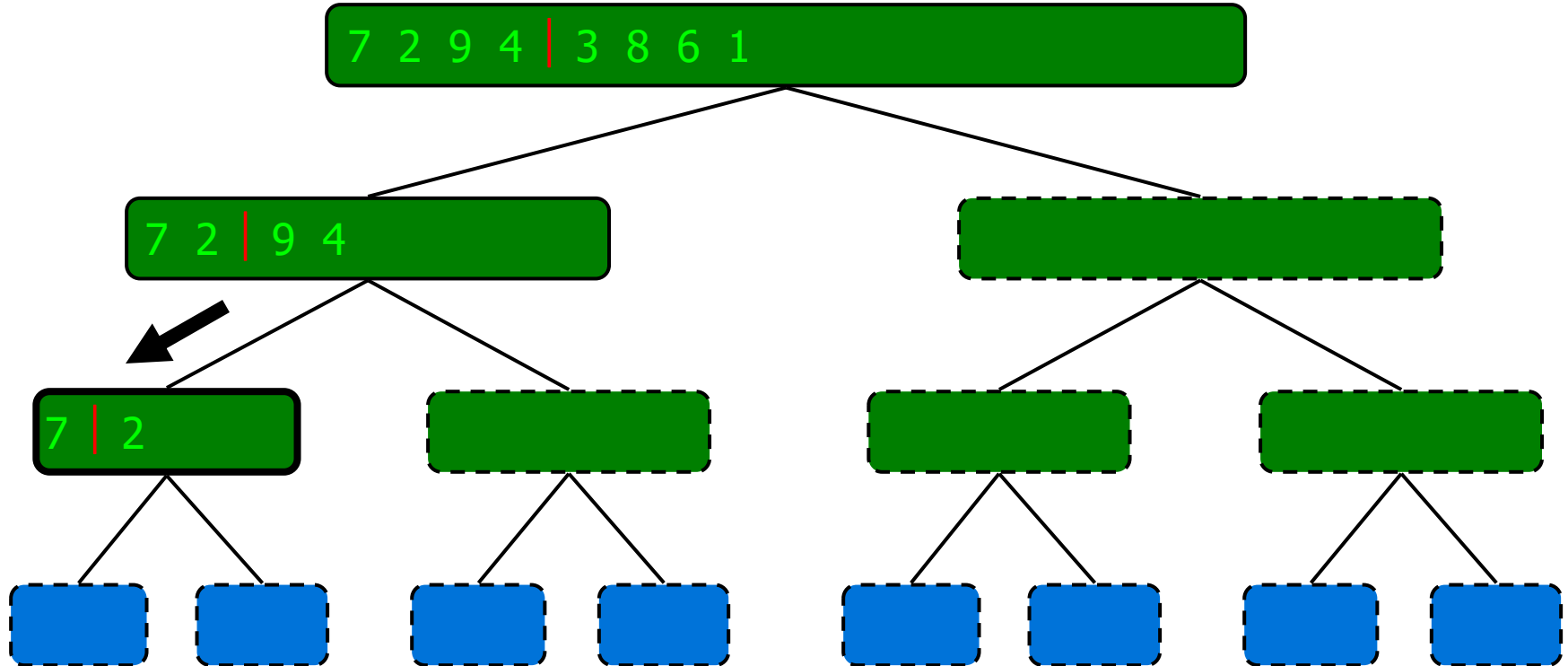
Execution Example (cont.)

- Recursive call, partition



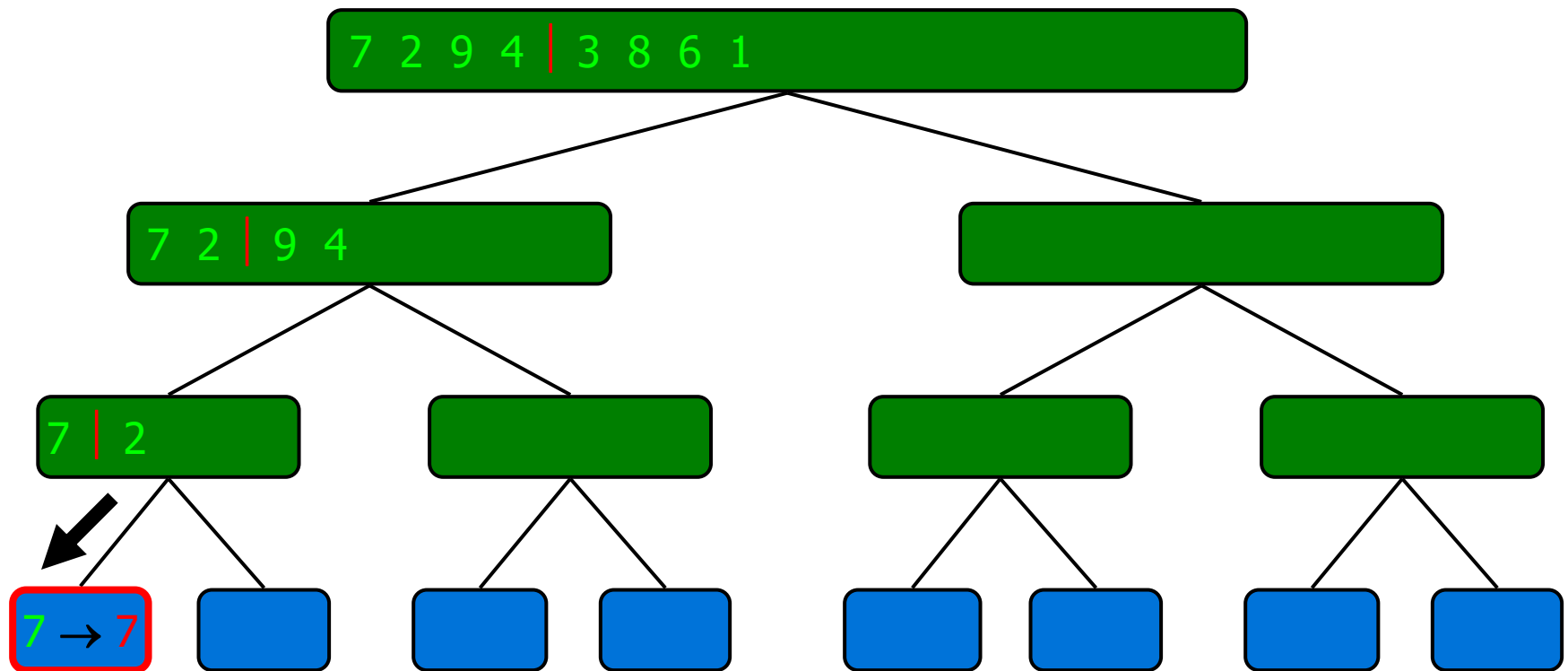
Execution Example (cont.)

- Recursive call, partition



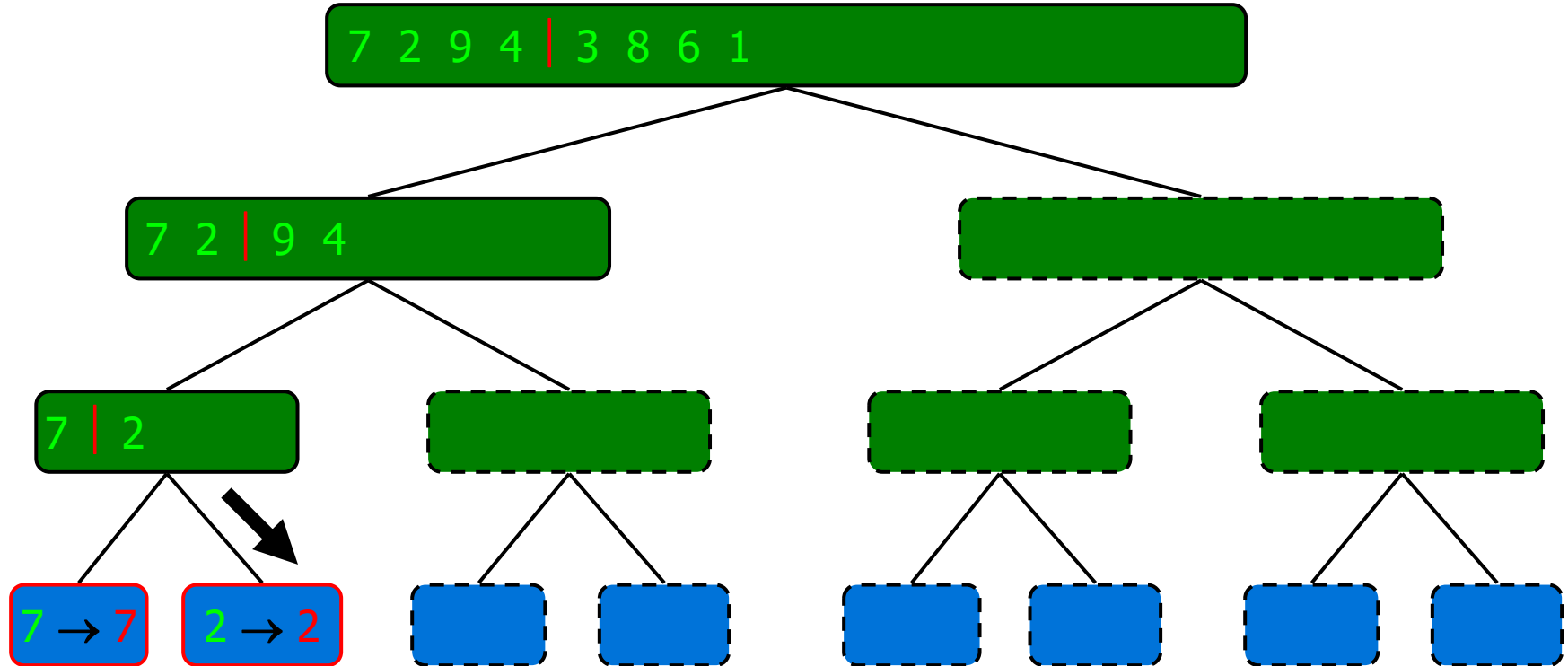
Execution Example (cont.)

- Recursive call, base case



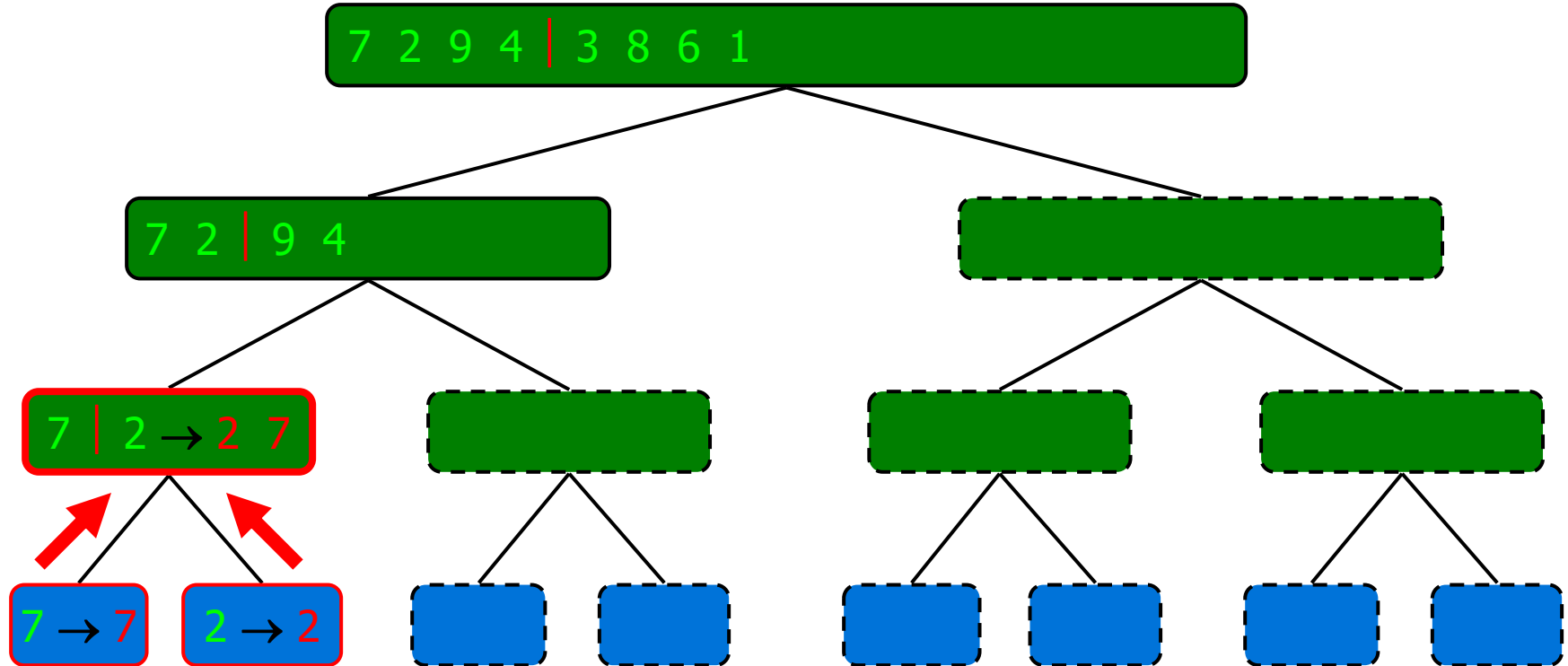
Execution Example (cont.)

- Recursive call, base case



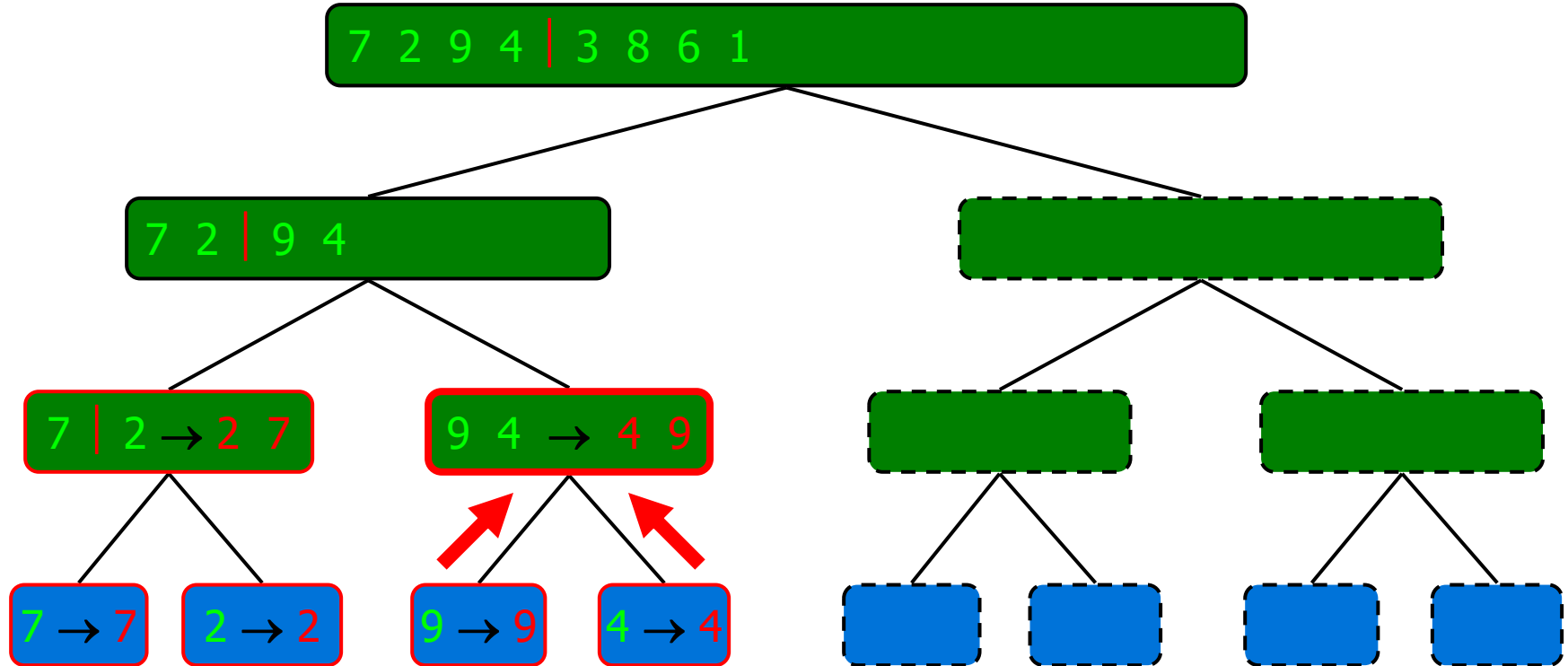
Execution Example (cont.)

- Merge



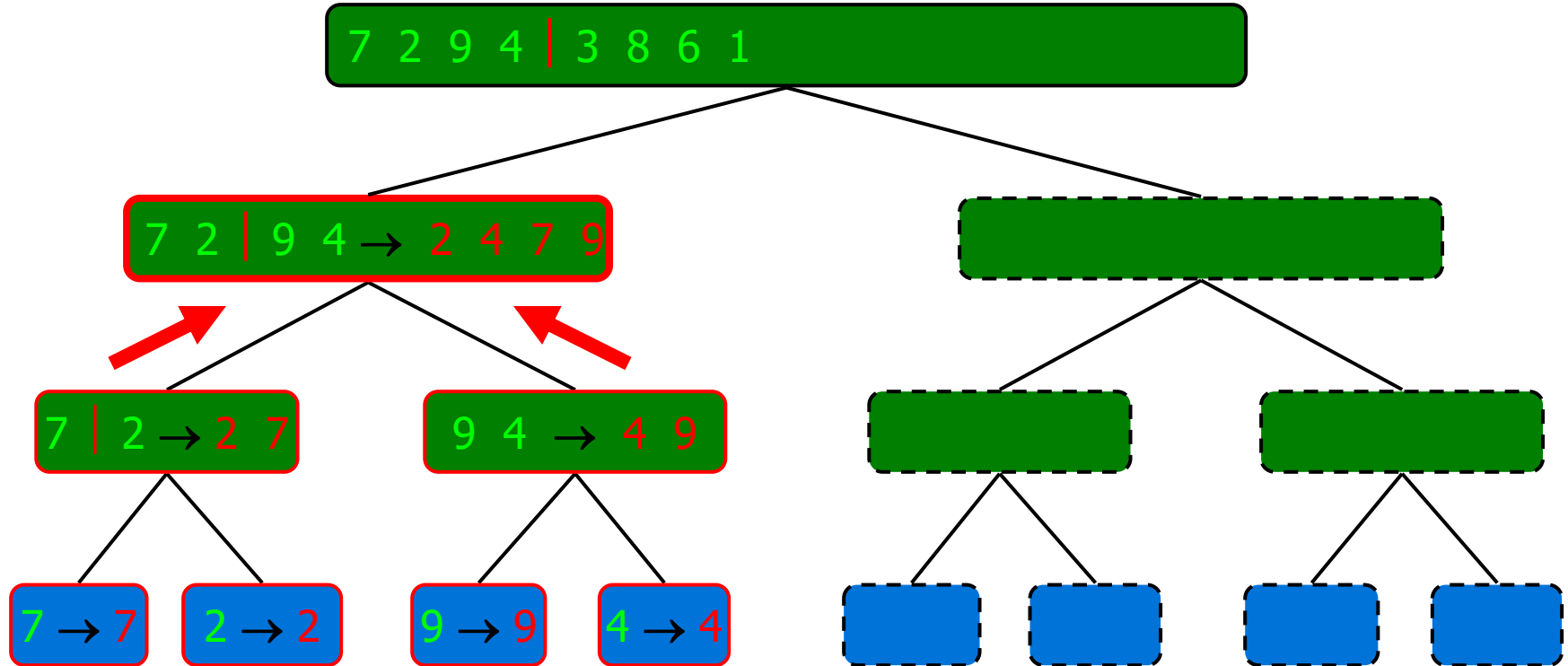
Execution Example (cont.)

- Recursive call, ..., base case, merge



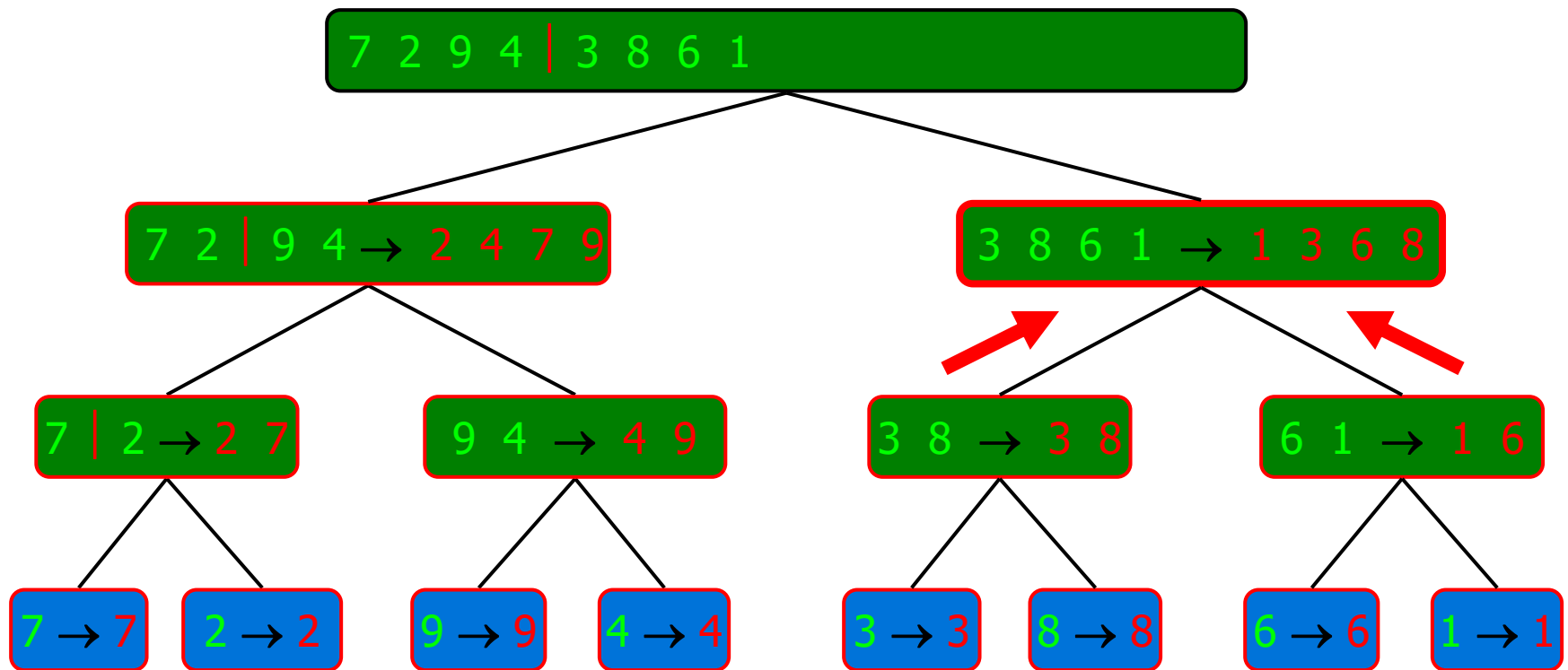
Execution Example (cont.)

■ Merge



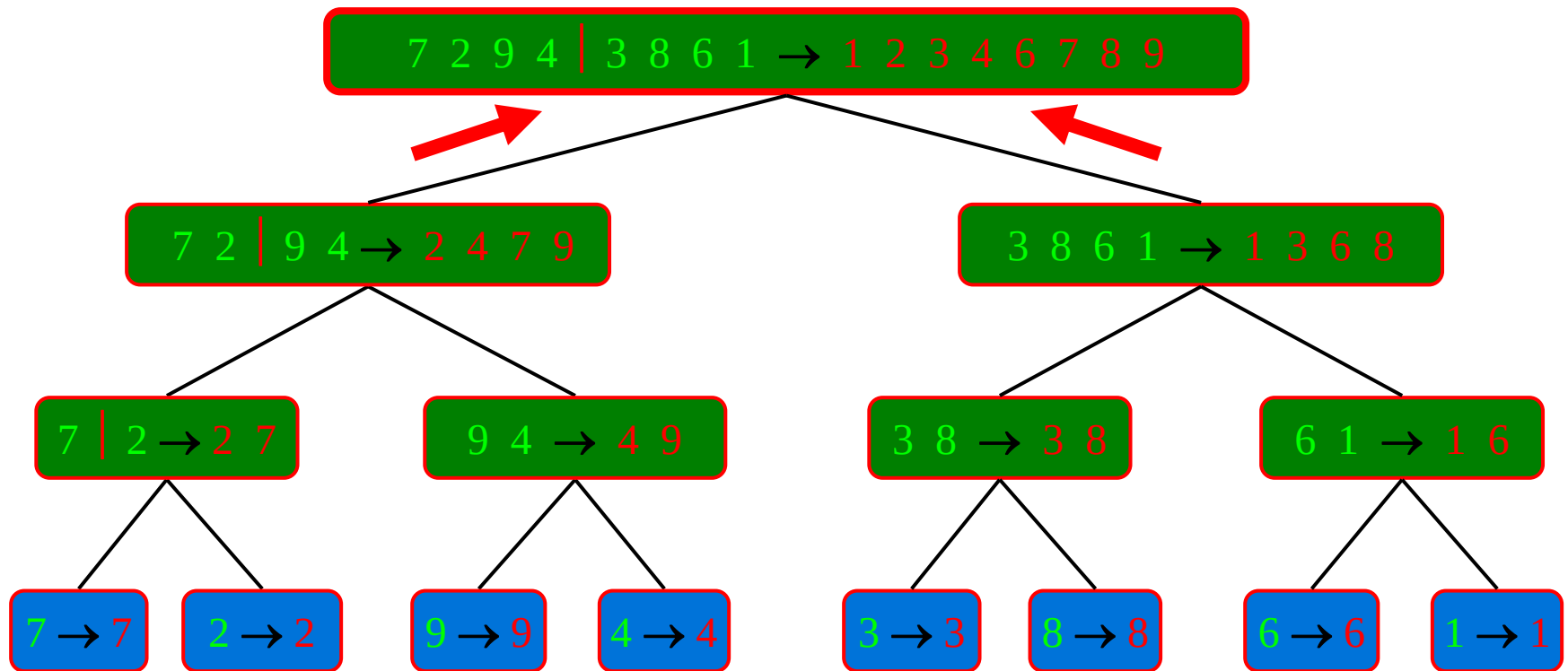
Execution Example (cont.)

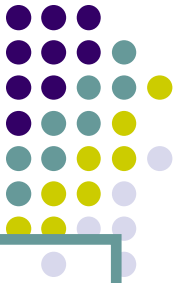
- Recursive call, ..., merge, merge



Execution Example (cont.)

■ Merge



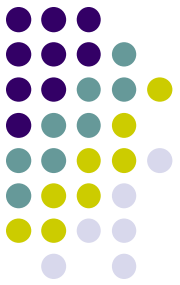


合并排序

 最坏时间复杂度: $O(n \log n)$

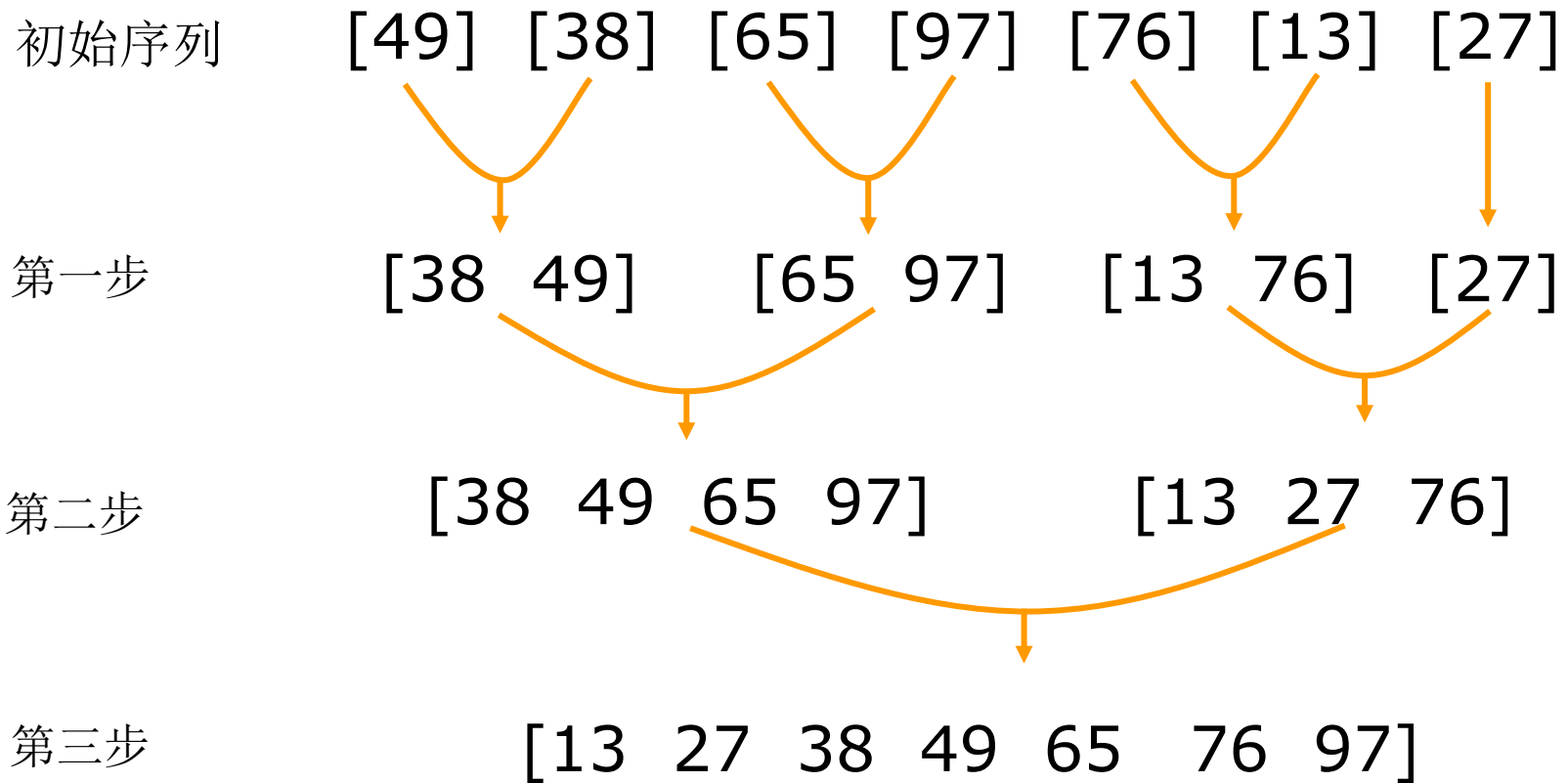
 平均时间复杂度: $O(n \log n)$

 辅助空间: $O(n)$



合并排序

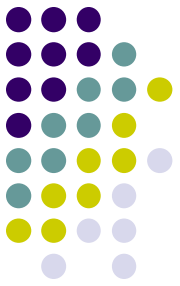
算法**mergeSort**的递归过程可以消去。





快速排序

快速排序



tudou.com

Quick Sort vs Bubble Sort

in a

10 Ball Sorting
Match



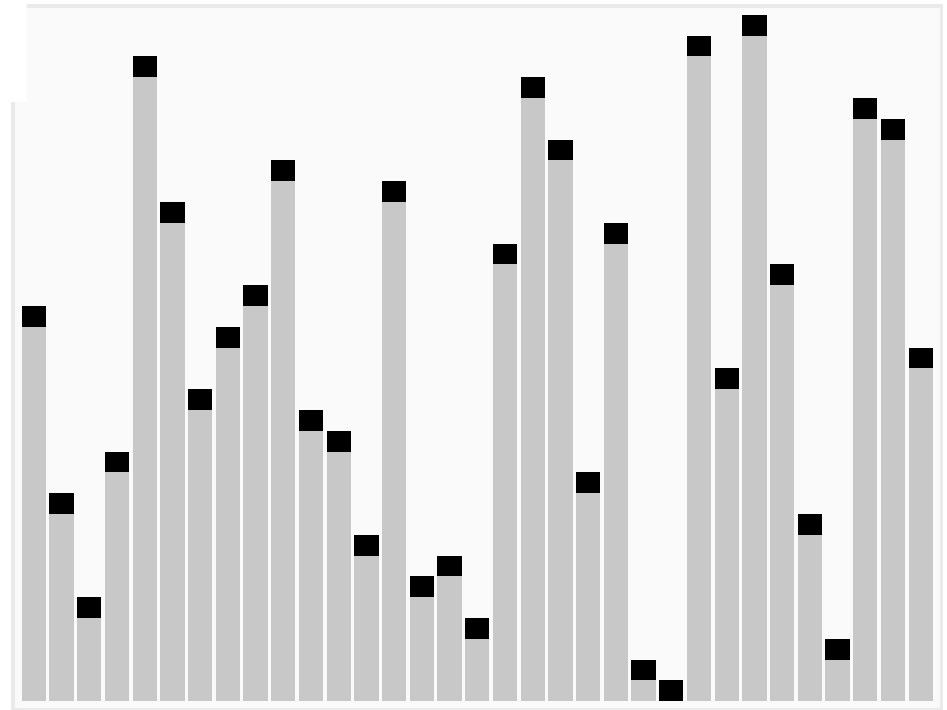
快速排序

记录的比较和交换
是从两端向中间进行的

```
template<class Type>
void QuickSort (Type a[], int p, int r)
{
    if (p < r) {
        int q = Partition(a, p, r);
        QuickSort (a, p, q-1);
        QuickSort (a, q+1, r);
    }
}
```

切分原问题

子问题



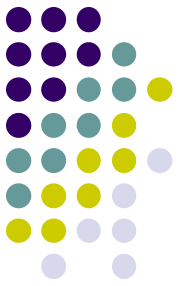
快速排序

分解过程

```
template<class Type>
int Partition
    (Type a[], int p, int r)
{
    int i = p, j = r + 1;
```

关键字较大的记录一次
就能交换到后面单元
关键字较小的记录一次
就能交换到前面单元

```
while (true) {
    while (a[++i] < x);
    // 将< x的元素交换到左边区
    域
    while (a[--j] > x);
    // 将> x的元素交换到右边区域
    if (i >= j) break;
    Swap(a[i], a[j]);
}
a[p] = a[j];
a[j] = x;
return j;
}
```



Example: Partition过程

哨兵

$\{6, 7, 5, 2, \bar{5}, 8\}$ 初始序列

$\{6, 7, 5, 2, \bar{5}, 8\}$ **j左移**

$\uparrow_i \quad \uparrow_j$

$\{\bar{5}, 7, 5, 2, 6, 8\}$ **i右移**

$\uparrow_i \quad \uparrow_j$

$\{\bar{5}, 6, 5, 2, 7, 8\}$ **j左移**

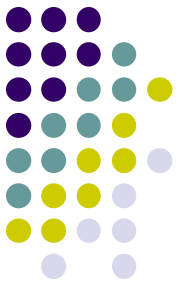
$\uparrow_i \quad \uparrow_j$

$\{\bar{5}, 2, 5, 6, 7, 8\}$ **i右移**

$\uparrow_i \quad \uparrow_j$

$\{\bar{5}, 2, 5\} 6 \{7, 8\}$ 完成

快速排序的进一步思考



- 快速排序算法的性能取决于划分的对称性。
- 通过修改算法**partition**，可以设计出采用**随机选择策略**的快速排序算法。

```
template<class Type>
int RandomizedPartition (Type a[], int p, int r)
{
    int i = Random(p,r);
    Swap(a[i], a[p]);
    return Partition (a, p, r);
}
```

随机选择划分基准



大整数乘法



大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆ 小学的方法: $O(n^2)$

✗ 效率太低

◆ 分治法:

X 复杂度分析

Y
$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

$T(n) = O(n^2)$ ✗ 没有改进

$$X = a 2^{n/2} + b \quad Y = c 2^{n/2} + d$$

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$

Tradition algorithm:

The diagram consists of several elements:

- A large 'X' symbol at the top left.
- A horizontal line below the 'X'.
- A staircase of asterisks (*) in the middle, starting from the left and moving towards the right, with each row having one more asterisk than the previous one.
- Another horizontal line below the staircase.
- A long row of asterisks at the bottom, spanning the width of the diagram.

 $\Theta(n^2)$ bit operations



大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆小学的方法： $O(n^2)$

✗效率太低

◆分治复杂度分析

XY

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^{\log_3 3}) = O(n^{1.59}) \quad \checkmark \text{较大的改进}$$

$$1. \quad XY = \underbrace{ac}_{bd} 2^n + ((\underbrace{a-c}_{b-d}) + \underbrace{ac+bd}_{bd}) 2^{n/2} +$$

细节问题：两个 XY 的复杂度都是 $O(n^{\log_3 3})$ ，但考虑到 $a+c, b+d$ 可能得到 $m+1$ 位的结果，使问题的规模变大，故不选择第2种方案。



大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆小学的方法: $O(n^2)$

✗效率太低

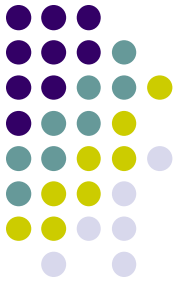
◆分治法: $O(n^{1.59})$

✓较大的改进

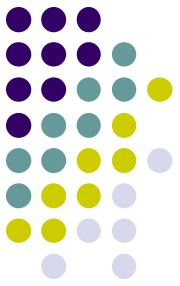
◆更快的方法??

➤如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。

➤最终的，这个思想导致了**快速傅利叶变换**(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。



Strassen矩阵乘法

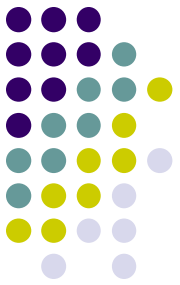


Strassen矩阵乘法

◆传统方法: $O(n^3)$

A和B的乘积矩阵C中的元素C[i,j]定义为:
$$C[i][j] = \sum_{k=1}^n A[i][k]B[k][j]$$

若依此定义来计算A和B的乘积矩阵C，则每计算C的一个元素C[i][j]，需要做n次乘法和n-1次加法。因此，算出矩阵C的 n^2 个元素所需的计算时间为 $O(n^3)$



Strassen矩阵乘法

◆传统方法： $O(n^3)$

◆分治法：

使用与
个大小

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 8T(n/2) + O(n^2) & n > 2 \end{cases}$$

$T(n) = O(n^3)$

成4

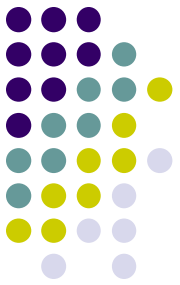
由此可得：

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$



Strassen矩阵乘法

◆传统方法: $O(n^3)$

◆分治法:

为了降低时间

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 7T(n/2) + O(n^2) & n > 2 \end{cases}$$

$$T(n) = O(n^{\log_2 7}) = O(n^{2.81}) \quad \checkmark \text{ 较大的改进}$$

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

$$M_3 = (A_{21} + A_{22})B_{11}$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

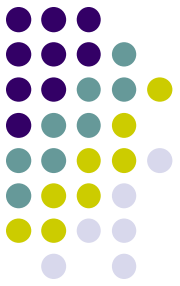


$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$



Strassen矩阵乘法

- ◆传统方法: $O(n^3)$
- ◆分治法: $O(n^{2.81})$
- ◆更快的方法??

➤Hopcroft和Kerr已经证明(1971), 计算2个 2×2 矩阵的乘积, 7次乘法是必要的。因此, 要想进一步改进矩阵乘法的时间复杂性, 就不能再基于计算 2×2 矩阵的7次乘法这样的方法了。或许应当研究 3×3 或 5×5 矩阵的更好算法。

➤在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性。目前最好的计算时间上界是 $O(n^{2.376})$

➤是否能找到 $O(n^2)$ 的算法?



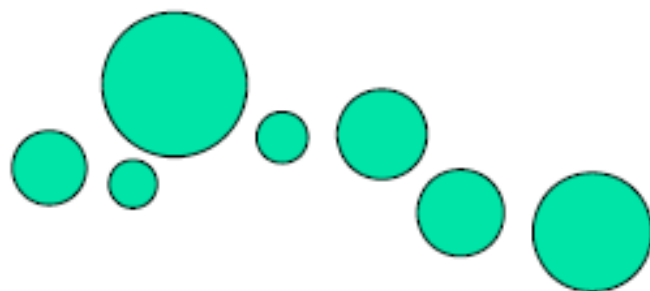
最大最小数问题



Exam4 Finding Minimum and Maximum (MaxMin)

- **Background**

Find the lightest and heaviest of n elements using a balance that allows you to compare the weight of 2 elements. (对于一个具有 n 个元素的数组，用一个天平，通过比较 2 个元素的重量，求出最轻和最重的一个)



- **Goal**

Minimize the number of comparisons. (正确找出需要的元素，最少的比较次数?)



Exam4 Finding Minimum and Maximum (MaxMin)

Max element

Find element with max weight (重量) from $w[0, n-1]$

```
maxElement=0
for (int i = 1; i < n; i++)
    if (w[maxElement] < w[i]) maxElement = i;
```

Number of comparisons (比较次数) is $n-1$.

- Obvious method (直接法)
 - ◆ Find the max of n elements making $n-1$ comparisons.
 - ◆ Find the min of the remaining $n-1$ elements making $n-2$ comparisons.
 - ◆ Total number of comparisons is $2n-3$.



Exam4 Finding Minimum and Maximum (MaxMin)

- Divide and conquer

Example

- ◆ Find the min and max of {3,5,6,2,4,9,3,1}.
 - $A = \{3,5,6,2\}$ and $B = \{4,9,3,1\}$.
 - $\min(A) = 2, \min(B) = 1$.
 - $\max(A) = 6, \max(B) = 9$.
 - $\min\{\min(A), \min(B)\} = 1$.
 - $\max\{\max(A), \max(B)\} = 9$.
- ◆ 选苹果; 挑运动员;

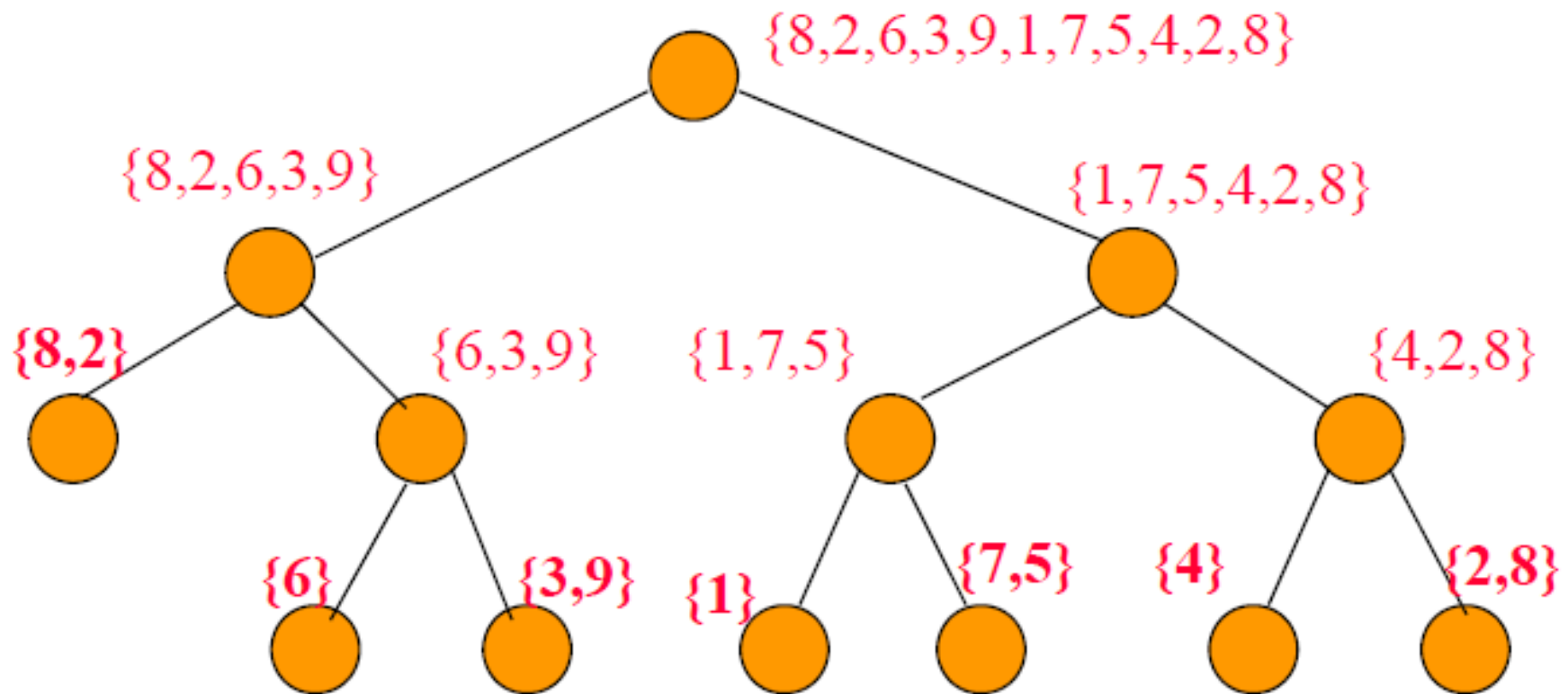


Exam4 Finding Minimum and Maximum (MaxMin)

- **Divide and conquer**

Example

- ◆ **Dividing Into Smaller Problems**



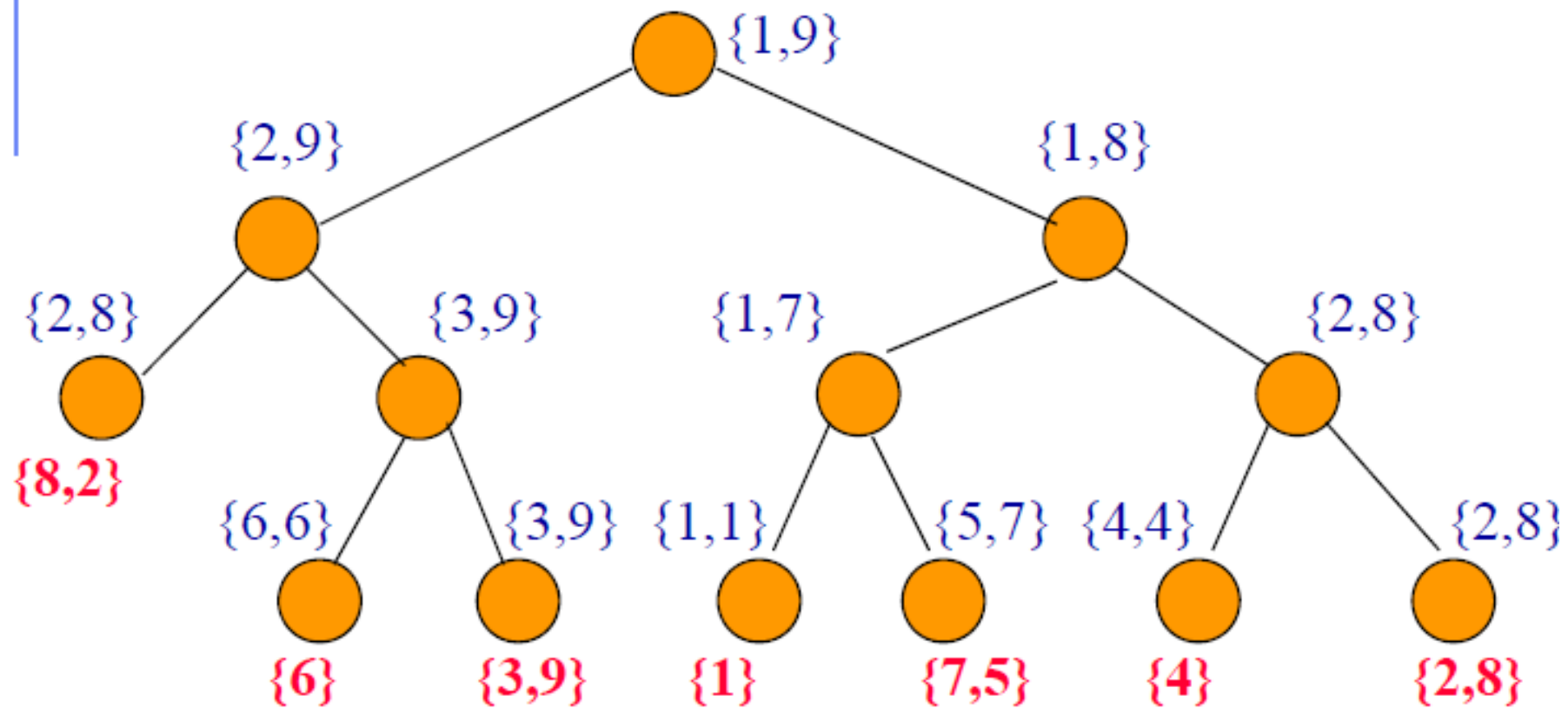


Exam4 Finding Minimum and Maximum (MaxMin)

- **Divide and conquer**

Example

- ◆ **Solve Small Problems and Combine**





Exam4 Finding Minimum and Maximum (MaxMin)

- Divide and conquer

MaxMin(L)

if length(L)=1 or 2, we use at most one comparison.

else

{ split (分裂) L into lists L1 and L2, each of $n/2$ elements

(min1, max1) = MaxMin(L1)

(min2, max2) = MaxMin(L2)

return (Min(min1, min2), Max(max1, max2))

}



Exam4 Finding Minimum and Maximum (MaxMin)

- Comparison between Obvious method ($2n-3$) and Divide-and-Conquer method ($3n/2-2$)

Assume that one comparison takes one second.

Time	$2n-3$	$3n/2-2$
1 minute	$n=31$	$n=41$
1 hour	$n=1801$	$n=2401$
1 day	$n=43201$	$n=57601$



多数问题



Exam5 Majority Problem (多数问题)

- Problem

Given an array A of n elements, only use “=” test to find the *majority element* (which appears more than $n/2$ times) in A .

- For example, given (2, 3, 2, 1, 3, 2, 2), then 2 is the majority element because $4 > 7/2$.

- Trivial solution:
counting (计数) is $O(n^2)$.

```
Majority(A[1, n])  
  for( $i = 1$  to  $n$ )  
     $M = 1$   
    for( $j = 1$  to  $n$ )  
      if ( $i \neq j$  and  $A[i] == A[j]$ )  $M++$   
    end  
    if ( $M > n/2$ ) return “A[i] is the majority”  
  end  
  return “No majority”
```



Exam5 Majority Problem (多数问题)

- Divide and conquer

Majority(A[1, n])

if $n=1$, then

return A[1]

else

m1=Majority(A[1, $n/2$])

m2=Majority(A[$n/2+1$, n])

test if m1 or m2 is the majority for A[1, n]

return majority or no majority.

Complexity analysis (Counting):

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

A=(2, 1, 3, 2, 1, 5, 4, 2, 5, 2)

f[1] = 2

f[2] = 4

f[3] = 1

f[4] = 1

f[5] = 2

However, there is a **linear time algorithm** for the problem.



```
for(i=1 to n) ++frequency[ A[i] ]
```

```
M = Max(frequency[ A[i] ] )
```

```
if (M > n/2)
```

```
check( M == frequency[ A[j] ] )
```

```
return "A[j] is the majority"
```

- Moral (寓意) of the story?



Exam5 Majority Problem (多数问题)

- **Divide and conquer**

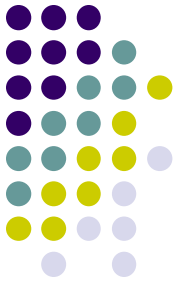
```
Majority(A[1, n])
if n=1, then
    return A[1]
else
    m1=Majority(A[1, n/2])
    m2=Majority(A[n/2+1, n])
    test if m1 or m2 is the majority for A[1, n]
    return majority or no majority.
```

**Complexity analysis
(Counting):**

$$T(n) = 2T(n/2) + O(n) \\ = O(n \log n)$$

However, there is a **linear time algorithm** for the problem.

- **Moral (寓意) of the story: Divide and conquer may not always give you the best solution!**

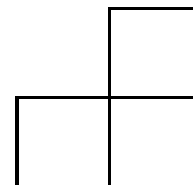
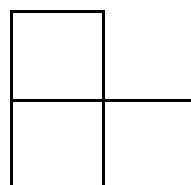
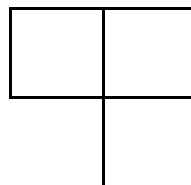
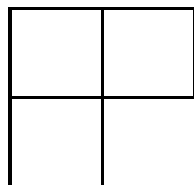
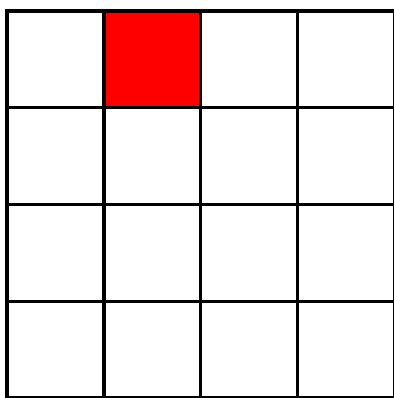


棋盘覆盖



棋盘覆盖

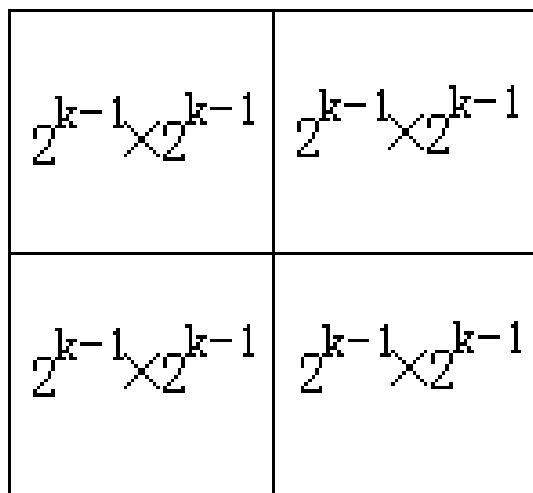
在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。



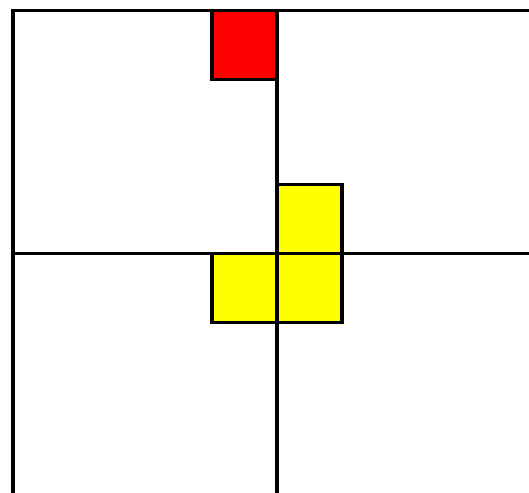


棋盘覆盖

当 $k > 0$ 时，将 $2^k \times 2^k$ 棋盘分割为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘(a)所示。特殊方格必位于4个较小子棋盘之一中，其余3个子棋盘中无特殊方格。为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，如(b)所示，从而将原问题转化为4个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为棋盘 1×1 。



(a)



(b)

棋盘覆盖



```
void chessBoard(int tr, int tc, int dr, int dc, int size)
```

```
{  
    board[tr + s - 1][tc + s] = t;  
    // 覆盖其余方格  
    chessBoard(tr, tc+s, tr+s-1, tc+s, s);  
    // 覆盖左下角子棋盘
```

```
// 覆盖左上角子棋盘
```

```
if (dr < tr + s
```

```
    // 特殊
```

```
    chess
```

```
else { // 用 t
```

```
    // 用 t
```

```
    board[tr
```

```
    // 覆盖其余方格
```

```
    chessBoard(tr, tc, tr+s-1, tc+s-1, s);
```

```
    // 覆盖右上角子棋盘
```

```
if (dr < tr + s && dc >= tc + s)
```

```
    // 特殊方格在此棋盘中
```

```
    chessBoard(tr, tc+s, dr, dc, s);
```

```
else { // 此棋盘中无特殊方格
```

```
    // 用 t 号 L 型骨牌覆盖左下角
```

复杂度分析

$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases}$$

$T(n) = O(4^k)$ 渐进意义下的最优算法

```
    // 覆盖右下角子棋盘
```

```
if (dr >= tr + s && dc >= tc + s)
```

```
    // 特殊方格在此棋盘中
```

```
    chessBoard(tr+s, tc+s, dr, dc, s);
```

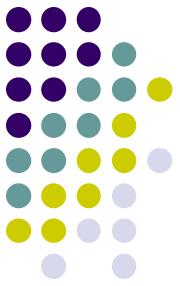
```
else { // 用 t 号 L 型骨牌覆盖左上角
```

```
    board[tr + s][tc + s] = t;
```

```
    // 覆盖其余方格
```

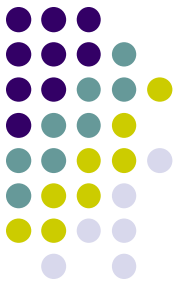
```
    chessBoard(tr+s, tc+s, tr+s, tc+s, s);
```

```
}
```



今日作业

- 课后习题2,4,5,6
- 上机题
 - 分别将插入排序、选择排序、合并排序和快速排序编程实现，并比较各种算法的效率。
 - 继续完成oj平台题目



今日任务

- 课后习题2-2, 2-4, 2-6
- 算法实现题1:
 - 在向量 $V[\text{first}, \text{last}]$ 中查找出第 K 大元素的值
- 算法实现题2:
 - 2-36